You said:

Treasure Hunt

Explore a procedurally generated dungeon, collect the key, then open the chest.

Getting Started

This game uses **pygame**. Install it before running:
pip install pygame-ce python game.py

> **Note:** If you are on Python 3.12 or newer, use `pygame-ce` (Community Edition) instead of `pygame`. It is a drop-in replacement with the same API.

Controls

Key	Action
W/A/S/D or Arrows	Move
R	Restart

Before changing the code

Play through the dungeon and observe how movement, tile collision, and item collection are handled. Trace the flow from `GameEngine` through `Player` and back. Understand how the grid and player state interact before making any modifications.

Task 1 — Traps

Add floor traps that send the player back to their starting position when stepped on.
Done when: the player is visibly reset to the start room on stepping a trap, and a clear status message is shown.

Task 2 — Enemy Guard

Add a guard that patrols back and forth near the chest. Touching the guard resets the player to start.
Done when: the guard moves between two patrol points each frame and correctly resets the player on collision.

Task 3 — Mini-map

Draw a small mini-map in a corner of the screen showing the dungeon layout.
Done when: the mini-map updates in real time, distinguishes walls from floor, and marks the player's current position.

Task 4 — Inventory UI

Show a HUD inventory slot that is empty at the start and displays a key icon once the key is collected.
Done when: the slot renders correctly in both states and updates immediately on key pickup.

Required testing

Collect the key then the chest, trigger every trap type, collide with the guard, verify the player resets correctly each time, check the mini-map reflects the actual grid, and confirm the inventory slot updates on collection. Test restarting mid-run.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated implementation.

Submission checklist

- Tasks 1–4 completed and tested.
- Traps reset the player correctly.
- Guard patrol and collision reset work.
- Mini-map reflects the dungeon in real time.
- Inventory slot updates on key pickup.
- No unnecessary external dependencies added beyond pygame.
- Code remains understandable and modular.
- Complete LLM chat-history link included.

Submission

Submission is only the following three things:

- A 10-second video of gameplay **before** your changes
- A 10-second video of gameplay **after** your changes, showing the new features working
- The Chat/LLM used page link, with the complete chat history

---

The above is my project task and strictly follow the above instructions and below i will add my game.py file change as mentioned above

game.py

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
SPEED = 3

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []
    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
```

```
            x = random.randint(1, COLS-w-1)
            y = random.randint(1, ROWS-h-1)
            room = pygame.Rect(x, y, w, h)
            overlap = any(room.inflate(2,2).colliderect(r) for r in rooms)
            if not overlap:
                rooms.append(room)
                for ry in range(y, y+h):
                    for rx in range(x, x+w):
                        grid[ry][rx] = FLOOR

        for i in range(len(rooms)-1):
            ax, ay = rooms[i].centerx, rooms[i].centery
            bx, by = rooms[i+1].centerx, rooms[i+1].centery
            cx = ax
            while cx != bx:
                grid[ay][cx] = FLOOR
                cx += 1 if bx > cx else -1
            cy = ay
            while cy != by:
                grid[cy][bx] = FLOOR
                cy += 1 if by > cy else -1

        if len(rooms) >= 2:
            cr, ck = rooms[-1], rooms[-2]
            grid[cr.centery][cr.centerx] = CHEST
            grid[ck.centery][ck.centerx] = KEY

        start = rooms[0] if rooms else None
        return grid, start

    COLORS = {
        WALL: (60,50,70),
        FLOOR: (200,190,170),
        CHEST: (200,160,30),
        KEY: (220,220,60),
    }

    class Player:
        def __init__(self, x, y):
            self.rect = pygame.Rect(x, y, 28, 28)
            self.color = (60,120,220)
```

```
        self.has_key = False

    def move(self, keys, grid, rows, cols):
        dx=dy=0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
        self._try_move(dx,0,grid,rows,cols)
        self._try_move(0,dy,grid,rows,cols)

    def _try_move(self, dx, dy, grid, rows, cols):
        new = self.rect.move(dx,dy)
        for px,py in [(new.left,new.top),(new.right-1,new.top),
(new.left,new.bottom-1),(new.right-1,new.bottom-1)]:
            c,r=px//TILE,py//TILE
            if not(0<=r<rows and 0<=c<cols) or grid[r][c]==WALL:
                return
        self.rect=new

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)
        if self.has_key:
            pygame.draw.circle(screen, (220,220,60), (self.rect.right-6,
self.rect.top+6), 5)


WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Treasure Hunt")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 24)
        self.big_font = pygame.font.SysFont("monospace", 40,
bold=True)
        self.reset()
```

```
    def reset(self):
        self.grid, start = generate_world()
        if start:
            sx = start.x * TILE + 6
            sy = start.y * TILE + 6
        else:
            sx, sy = TILE+6, TILE+6
        self.player = Player(sx, sy)
        self.won = False
        self.status = "Find the KEY, then the CHEST!"

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r: self.reset()
        return True

    def update(self):
        if self.won: return
        keys = pygame.key.get_pressed()
        self.player.move(keys, self.grid, ROWS, COLS)
        pr = self.player.rect.centery // TILE
        pc = self.player.rect.centerx // TILE
        if 0<=pr<ROWS and 0<=pc<COLS:
            cell = self.grid[pr][pc]
            if cell == KEY:
                self.player.has_key = True
                self.grid[pr][pc] = FLOOR
                self.status = "Got the key! Find the CHEST!"
            elif cell == CHEST and self.player.has_key:
                self.won = True
                self.status = "Treasure found!"

    def draw(self):
        self.screen.fill((30,25,40))
        for r in range(ROWS):
            for c in range(COLS):
                cell = self.grid[r][c]
                rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
```

```
                pygame.draw.rect(self.screen, COLORS[cell], rect)
                if cell == KEY:
                    pygame.draw.circle(self.screen, (255,240,60),
(c*TILE+TILE//2, r*TILE+TILE//2),10)
                elif cell == CHEST:
                    pygame.draw.rect(self.screen,
(180,120,20),rect.inflate(-12,-12),border_radius=4)
        self.player.draw(self.screen)
        hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
        pygame.draw.rect(self.screen,(20,20,35),hud)
        st = self.font.render(self.status+" | R=Restart", True,
(200,200,200))
        self.screen.blit(st,(8,ROWS*TILE+13))
        if self.won:

ov=pygame.Surface((WIDTH,ROWS*TILE),pygame.SRCALPHA)
            ov.fill((0,0,0,140))
            self.screen.blit(ov,(0,0))
            msg=self.big_font.render("TREASURE FOUND!", True,
(220,180,30))
            sub=self.font.render("Press R to Play Again",True,
(180,180,180))
            self.screen.blit(msg,(WIDTH//2-
msg.get_width()//2,ROWS*TILE//2-30))
            self.screen.blit(sub,(WIDTH//2-
sub.get_width()//2,ROWS*TILE//2+20))
        pygame.display.flip()

    def run(self):
        running=True
        while running:
            running=self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()
```

Analyzed

You said:

more better make it
crazyyy

The updated file passes Python syntax compilation. Please run it locally to verify the new visual effects and all four required tasks.

My next upgrade recommendation: a proper rogue-like experience with animated torches, multiple enemy types, health hearts, a dash ability, sound effects, boss-room lighting, and a dramatic treasure-opening animation — while preserving your lab requirements and keeping the project in a single Python file.

‹/›

You said:

no make sure there is a way to win the game and give like big map lets goo

Analyzed

Analyzed

Analyzed

ChatGPT said:

LET'S GOOOOOO 🔥 🔥 The dungeon is now a **much larger, scrolling world**, and I've adjusted the generation so the key and chest are reachable!

Download the new big-map version

Treasure Hunt: Big Dungeon

32 × 22 tiles · Scrolling camera · Guaranteed connected dungeon layout

Download the updated game.py ↗

What's improved?

Bigger world

704 dungeon tiles, with the camera following you as you explore.

Connected rooms

Corridors connect the rooms so you can reach the objective.

Winnable objective

Full-map mini-map

Find the key, then travel to the chest. The key stays with you after a trap or guard hit.

See the entire dungeon layout and track your position.

Run it

```
python -m pip install pygame-ce
python game.py
```

Your objective: explore the dungeon, collect the golden key, avoid the traps and guard, and unlock the treasure chest.

One honest note: I've checked the updated file for Python syntax errors, but this environment doesn't have Pygame installed, so I couldn't run a live gameplay test here. Please test the game locally before submitting it.

‹/›